

Introduction to PYTHON

1. What are the types of applications ?

Ans : Python is heavily used for data analysis, data visualization, and statistical modeling. Common libraries include:

- Pandas – For data manipulation and analysis.
- NumPy – For numerical computing and array manipulation.
- Matplotlib, Seaborn – For data visualization.
- SciPy – For scientific and technical computing.
- Jupyter Notebooks – For interactive computing and data analysis.

2. What is programming ?

Ans : Programming is the process of writing code to give instructions to a computer to perform specific Tasks.

3. What is PYTHON ?

Ans : Python is a high-level, easy-to-read, and versatile programming language used for web development, data science, automation, AI, and more. It is known for its simplicity and large community support.

4. How memory is managed in Python?

Ans : Memory management in Python is handled automatically using the following mechanisms:

1. Reference Counting – Every object has a reference count, and when it reaches zero, the object is deleted.
2. Garbage Collection (GC) – Python has an automatic garbage collector that removes unused objects to free memory.
3. Dynamic Memory Allocation – Python's memory manager allocates and manages memory dynamically.
4. Memory Pools (PyMalloc) – Python uses private heaps and memory pools to optimize memory allocation.

5. What is the purpose continuing statement in python ?

Ans : Purpose of continue

1. Skipping certain values in a loop
2. Avoiding unnecessary computations
3. Improving readability and efficiency

6. What are negative indexes and why are they used ?

Ans :

1. Access elements from the end without needing the length
2. Simplifies code for reverse traversal
3. Useful for slicing lists or strings from the end

7. What is List? How will you reverse a list ?

Ans : Python provides multiple ways to reverse a list :

1. "reverse()" modifies the original list.
2. "reversed()" provides an iterator, useful for iterating without making a copy.

7. How will you remove last object from a list ?

Ans : Using pop() Method : The pop() method removes and returns the last element of the list.

8. Differentiate between append () and extend () methods ?

Ans :

append()	extend()
Adds a single element (or an object) to the end of the list.	Adds elements from an iterable (list, tuple, set, etc.) to the list.
The entire object is added as a single item.	Each element from the iterable is added individually.
Modifies the list in place, returns None.	Modifies the list in place, returns None.
When you want to add a single item (int, string, list, etc.).	When you want to add multiple items from an iterable.

9. How will you compare two lists ?

Ans : 1) The == operator checks if both lists have the same elements in the same order.

2) Using sorted() If order doesn't matter, sorting both lists before comparison helps.

3) Using set() If duplicates don't matter and only unique elements need comparison

4) Using collections.Counter() To check if both lists contain the same elements with the same Frequency.

10. What is tuple? Difference between list and tuple.

Ans : Tuple vs. List (Short Version)

Tuple (tuple):

- Immutable (cannot be changed)
- Faster & uses less memory
- Defined with ()
- Example: my_tuple = (1, 2, 3)

List (list):

- Mutable (can be changed)

- Slower & uses more memory
- Defined with []
- Example: my_list = [1, 2, 3]

11. How will you create a dictionary using tuples in python?

Ans : You can create a dictionary from tuples using the dict() function, where each tuple represents a key-value pair.

12. How Do You Traverse Through a Dictionary Object in Python ?

Ans : 1) Traverse Keys (Default)

2) Traverse Keys & Values (items())

3) Traverse Only Values (values())

13. How to Check if a Key Exists in a Dictionary in Python ?

Ans : To check if a key exists in a dictionary in Python, the most efficient way is to use the in operator, as in if "age" in my_dict, which quickly checks for key presence. Another approach is using the get() method, where if my_dict.get("age") is not None ensures safe key lookup without raising a KeyError. While you can also use if "age" in my_dict.keys(), it is less efficient because keys() creates a separate list of keys. Overall, the in operator is the best choice for checking key existence due to its speed and simplicity.

14. How Many Basic Types of Functions Are Available in Python ?

Ans : Python provides four basic types of functions:

1. Built-in Functions – Predefined functions like print(), len(), type(), and sum().
2. User-defined Functions – Functions created by users using the def keyword.
3. Lambda (Anonymous) Functions – Small, single-expression functions using lambda, e.g., lambda x: x * 2.
4. Recursive Functions – Functions that call themselves, useful for problems like factorial and Fibonacci series.

15 . How can you pick a random item from a list or tuple?

Ans : To pick a random item from a list or tuple in Python, you can use the random.choice() function from the random module. This method randomly selects a single item from the given sequence, whether it's a list or a tuple. For example, random.choice(my_list) will return a random element from my_list. If you need to select multiple items, random.choices() allows for selecting items with repetition, while random.sample() is used when you want to pick multiple unique items without repetition. These functions make it simple and efficient to handle random selection in Python.

16. How can you pick a random item from a range ?

Ans : To pick a random item from a range in Python, you can use the random.choice() function in combination with the range() function. First, define a range of numbers using range(), such as range(1, 11) for numbers 1 through 10. Then, use random.choice() to randomly select an item from that range. This approach allows you to efficiently pick a random number from a specific sequence or range of numbers. It's simple and works for any range you define.

17. How can you get a random number in python?

Ans : To generate a random number in Python, you can utilize the random module. For generating a random integer between a specific range, use `random.randint(a, b)`. To get a random floating-point number within a range, `random.uniform(a, b)` can be used. If you need a float between 0 and 1, `random.random()` will provide that. Additionally, `random.choice(list)` allows you to select a random item from a list. These methods make it easy to generate random numbers or choose random elements in Python, depending on your requirements.

18. How will you set the starting value in generating random numbers ?

Ans : To set the starting value (or seed) for generating random numbers in Python, you can use the `random.seed()` function. This function initializes the random number generator with a specific seed, ensuring that the sequence of random numbers is reproducible. For example, if you set `random.seed(42)`, the same random numbers will be generated every time you run the code, allowing you to achieve consistent results. Setting a seed is particularly useful for debugging or testing, as it ensures that the output is predictable. Without setting a seed, Python uses the system time as the default seed, resulting in different random numbers each time.

19. How will you randomize the items of a list in place?

Ans : To randomize the items of a list in place, you can use the `random.shuffle()` function from the random module. This function directly modifies the original list by rearranging its elements in a random order. For example, calling `random.shuffle(my_list)` will shuffle the items of `my_list` without returning a new list. It's a simple and efficient way to randomize the list, ensuring that the changes are applied to the original list itself, rather than creating a separate shuffled version.

20 . What is File function in python? What are keywords to create and write file.

Ans : In Python, file handling is done through built-in functions, with the primary function being `open()`, which allows you to interact with files in various modes. The `open()` function opens a file and returns a file object that can be used for reading, writing, or appending data. To create or write to a file, you can use the 'w' mode, which creates a new file or overwrites an existing file. If you want to add content to an existing file without overwriting it, you can use the 'a' (append) mode. Alternatively, the 'x' mode is used for creating a new file but raises an error if the file already exists. After writing or appending to a file, it's important to close the file using the `close()` method to release resources, though the `with` statement is often used to automatically handle file closing after the block of code is executed. Functions like `write()` and `writelines()` are used to insert content into a file. These keywords and modes make file creation, writing, and handling straightforward and efficient in Python .

21. Explain Exception handling? What is an Error in Python?

Ans : Exception handling in Python is a mechanism that allows you to handle errors gracefully during program execution. Instead of allowing the program to crash when an error occurs, you can catch and manage exceptions using `try`, `except`, `else`, and `finally` blocks. The `try` block contains code that may raise an exception, while the `except` block handles the exception if it occurs. The `else` block runs if no exceptions are raised, and the `finally` block ensures that certain code executes regardless of whether an exception occurred, such as resource cleanup. Errors in Python can be classified into two types: syntax errors, which occur when the code doesn't follow proper syntax rules and are detected during compilation, and exceptions, which are runtime errors that can be caught and handled during execution. Common

exceptions include `ZeroDivisionError`, `TypeError`, and `FileNotFoundError`. Exception handling provides a structured way to deal with errors and keep the program running smoothly.

22. How many except statements can a try-except block have? Name Some built-in exception classes:

Ans : A try-except block in Python can have multiple except statements to handle different types of exceptions. You can specify a different except block for each exception type, allowing you to handle various errors in distinct ways. The except blocks are checked in the order they are written, and the first one that matches the raised exception will be executed. Additionally, you can have a generic except block to catch any exception that wasn't explicitly handled by the other except blocks.

23. When will the else part of try-except-else be executed?

Ans : The else part of a try-except-else block in Python is executed only if no exception is raised during the execution of the code in the try block. If an exception occurs in the try block, the program will immediately jump to the appropriate except block, and the else block will be skipped. However, if no exception is raised, the code in the else block will run. This is useful for situations where you want to execute certain code only if the try block was successful and no error occurred. The else block helps to separate the code that should run when no errors occur from the error-handling logic in the except block, making the code cleaner and easier to understand.

24. Can one block of except statements handle multiple exception ?

Ans : yes, a single except block in Python can handle multiple exceptions. You can specify multiple exception types within a single except block by enclosing them in parentheses, separated by commas.

25. When is the finally block executed ?

Ans : The finally block in Python is executed no matter what—whether an exception is raised or not, and regardless of whether the exception is caught or not. It is typically used for clean-up actions that need to occur after the try block execution, such as closing files, releasing resources, or performing any other necessary finalization steps. Even if the program encounters an error in the try block and the exception is caught in the except block, the finally block will still be executed.

26. What happens when „1“== 1 is executed ?

Ans : When the expression `'1' == 1` is executed in Python, it will evaluate to `False`. This is because Python is comparing two objects of different types: the left-hand side is a string (`'1'`), and the right-hand side is an integer (`1`).

27. How Do You Handle Exceptions with Try/Except/Finally in Python? Explain with coding snippets.

Ans : In Python, exception handling is done using try, except, and finally blocks. The try block contains code that might raise an exception. If an error occurs, the except block handles the exception. The finally block runs no matter what, making it ideal for cleanup tasks like closing files. You can also use the else block, which executes only if no exception occurs in the try block. This structure allows you to manage errors gracefully and ensure that necessary actions are always taken, even if an exception occurs. Using try, except, and finally blocks allows you to handle errors gracefully in Python. The try block contains the code that might raise an exception, the except block handles the exception if it occurs, and the finally block ensures that certain code runs regardless of exceptions, often for cleanup purposes. The else block can

also be used to run code when no exceptions are raised. This structure ensures that your program can handle errors without crashing and that resources are properly managed.